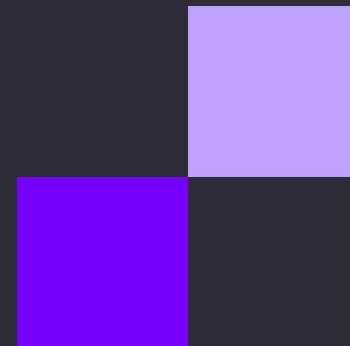




онлайн
университет



ВЫСШАЯ ШКОЛА
ЭКОНОМИКИ



SQL для начинающих

Занятие 4



Операторы CASE WHEN

```
CASE  
    WHEN condition_1 THEN result_1  
    WHEN condition_2 THEN result_2  
    [WHEN ...]  
    [ELSE else_result]  
END
```

Операторы CASE WHEN

```
SELECT
    payment_id,
    amount_paid,
    CASE
        WHEN amount_paid > 50000 THEN 'Важный клиент'
        WHEN amount_paid > 10000 THEN 'Обычный клиент'
        ELSE 'Невыгодный клиент'
    END AS category
FROM payments
```

Операторы CASE WHEN

```
SELECT
    room_number,
    price_per_night,
    CASE
        WHEN price_per_night > 5000 THEN 'Дорогой номер'
        WHEN price_per_night > 2500 AND price_per_night <= 5000 THEN 'Нормальный номер'
        ELSE 'Дешевый номер'
    END AS room_price_category
FROM rooms
```

Группировка и агрегация данных

Агрегация — это способ преобразования набора данных в одно результирующее значение. Типичные примеры — подсчет среднего значения, медианы, суммы всех значений ит.д.

Приведем пример наиболее популярных агрегирующих функций в SQL:

- COUNT(*) — подсчет количества строк в результирующей выборке
- COUNT() — подсчет не пропущенных значений в столбце выборки (тех значений, которые не являются NULL)
- AVG() — среднее арифметическое по всем значениям столбца выборки .
- SUM() — сумма значений столбца выборки
- MIN() — выбор минимального значения в столбце в выборке
- MAX() — выбор максимального значения в столбце в выборке

Группировка и агрегация данных

Посмотрим, сколько всего клиентов записано в нашей БД:

```
SELECT COUNT(*) FROM clients;
```

Группировка и агрегация данных

Посмотрим, сколько всего клиентов записано в нашей БД:

```
SELECT COUNT(*) FROM clients;
```

И сразу рассмотрим разницу операторов COUNT со звездой и с колонкой. Посчитаем, сколько у нас всего клиентов в базе и у скольких из них сохранены телефон и электронная почта:

```
SELECT  
    COUNT(*) as all_count,  
    COUNT(email) as email_count,  
    COUNT(phone_number) as phone_count  
FROM clients;
```

Группировка и агрегация данных

Посмотрим, когда прошла самая первая оплата:

```
SELECT MIN(payment_date) AS min_payment_dt  
FROM payments;
```


Группировка и агрегация данных

Чаще всего нам интересны статистики не по полной таблице, а по каким-то группам данных. Например, мы можем захотеть узнать, сколько денег получила наша гостиница по каждому из дней. Для создания групп используется оператор GROUP BY:

```
SELECT SUM(amount_paid)
FROM payments
GROUP BY payment_date;
```

Группировка и агрегация данных

Конечно, чаще нас интересует информация по конкретным дням: например, за месяц. В этом случае можем использовать GROUP BY совместно с WHERE. Порядок операторов в запросе важен!

```
SELECT SUM(amount_paid)
FROM payments
WHERE payment_date BETWEEN '2018-01-01' AND '2018-01-31'
GROUP BY payment_date;
```

HAVING

Оператор WHERE умеет работать только со статичными условиями — теми, которые не надо считать на ходу. Если мы хотим отфильтровать группы по значению результата выполнения агрегирующей функции, то следует использовать оператор HAVING.

HAVING в SQL используется для фильтрации результатов агрегатных функций в группах данных, созданных с помощью GROUP BY. Он позволяет задать условия, применяемые после выполнения группировки. Это похоже на WHERE, но WHERE действует до агрегирования, а HAVING — после.

```
SELECT столбец, агрегатная_функция(столбец)  
FROM таблица  
GROUP BY столбец  
HAVING условие;
```

HAVING

Выберем дни, в которые выручка была более 30000 рублей:

```
SELECT SUM(amount_paid) AS income  
FROM payments  
GROUP BY payment_date  
HAVING SUM(amount_paid) > 30000;
```

HAVING

Эту конструкцию также можно использовать и при обычном условии WHERE – например, отфильтруем те же прибыльные дни, но за период:

```
SELECT SUM(amount_paid) AS income
FROM payments
WHERE payment_date BETWEEN '2018-01-01' AND '2018-01-31'
GROUP BY payment_date
HAVING SUM(amount_paid) > 30000;
```

HAVING

Важно отметить, что даже запросы с таким большим количеством условий и группировок можно сортировать:

```
SELECT SUM(amount_paid) AS income
FROM payments
WHERE payment_date BETWEEN '2018-01-01' AND '2018-01-31'
GROUP BY payment_date
HAVING SUM(amount_paid) > 30000;
ORDER BY income DESC
```

Задача 1

Найдите среднюю цену за ночь для всех номеров

Задача 1

Найдите среднюю цену за ночь для всех номеров

```
SELECT AVG(price_per_night) AS avg_price  
FROM rooms
```


Задача 2

Найдите количество номеров, которые имеют площадь менее 30 квадратных метров

Задача 2

Найдите количество номеров, которые имеют площадь менее 30 квадратных метров

```
SELECT COUNT (room_size_sqm) AS cnt_rooms  
FROM rooms  
WHERE room_size_sqm < 30
```

Задача 3

Найдите количество бронирований, которые были оплачены наличными

Задача 3

Найдите количество бронирований, которые были оплачены наличными

```
SELECT COUNT (payment_method) AS cnt_cash  
FROM payments  
WHERE payment_method = 'наличными'
```

Задача 4

Найдите клиентов, которые сделали более 3 бронирований и выведите их с указанием количества бронирований

Задача 4

Найдите клиентов, которые сделали более 3 бронирований и выведите их с указанием количества бронирований

```
SELECT renter_id, COUNT(*) AS booking_count  
FROM bookings  
GROUP BY renter_id  
HAVING COUNT(*) > 3
```

Задача 5

Найдите среднюю сумму оплаты для каждого способа оплаты

Задача 5

Найдите среднюю сумму оплаты для каждого способа оплаты

```
SELECT payment_method, AVG(amount_paid) AS avg_payment  
FROM payments  
GROUP BY payment_method
```


Спасибо за внимание!